BRAIN

Research Graph Engine

Comprehensive Technical Reference

Architecture · Data Flow · Module Internals · Algorithms

Document Type	Internal Technical Reference
Codebase	Brain / Research Graph Engine
Language	Python 3, C++17
Key Libraries	spaCy, sentence-transformers, reportlab, FastAPI, Three.js
Date	May 2026

Table of Contents

1. System Overview

1.1 Introduction and Purpose

Brain is a research graph engine that autonomously transforms a pair of natural-language topic queries into a structured knowledge graph, then traverses that graph to synthesize a coherent analytical report. It combines web-scraping, NLP-driven clause extraction, semantic embedding, a real-time spring physics simulation, and a C++ thought-traversal engine to discover and articulate relationships between arbitrary concepts.

The system is designed to operate entirely locally: all embedding and language-model inference runs against a local sentence-transformers model and a locally-hosted Ollama instance (llama3.2:3b by default). External network access is limited to Serper.dev search API calls and HTTP page fetches during the research phase.

1.2 High-Level Architecture

The codebase is partitioned into five conceptual layers, each with its own filename prefix convention:

Prefix / Layer	Responsibility
u_ (utilities/config)	Frozen constants, NLP token lists, shared helper functions.
d_ (data ingestion)	Scraping, extraction, NLP parsing, semantic storage. All I/O-heavy workers.
o_ (object model)	Pure Python data classes: ConnectionEndpoint, Connector, Info_Agent, Thought.
p_ (graph/physics)	In-memory graph state, SHM management, C++ subprocess wrappers, active subgraph pruning.
s_ (synthesis)	Argument selection, LLM report generation, result export.
frontend/	FastAPI dashboard server, Three.js 3D visualiser, HTML shell.

1.3 Data Flow Summary

A research session follows a strict linear pipeline triggered by a user command containing two topic targets (A and B). The stages are:

- **Command receipt** — User types targets A and B in the dashboard; the FastAPI server writes a JSON command into the COMMAND shared memory segment.
- **Query expansion** — m_runtime.py wakes up, calls d_query_expander.build_query_plan() to produce 15 search query strings across three categories: target-A focused, target-B focused, and bridging queries.
- **Scraping** — Up to SCRAPE_THREADS=8 parallel scrape workers fetch Serper.dev results + Wikipedia; raw text blocks are placed on extract_queue.
- **Extraction** — Up to EXTRACTION_THREADS=4 extraction workers dequeue blocks, run d_logic_extractor.find_connections(), and push connection records onto asu_queue.
- **Graph ingestion** — m_runtime ingests from asu_queue; p_connection_graph.add_connection() resolves agent names via cosine similarity, packs 40-byte binary records into SHM, and attaches Python Connector objects.

- **Refinement scrape** — If fewer than a threshold of connections are found, a second wave of scrapes is launched with refined queries derived from the most-connected agents found so far.
- **Physics simulation** — The C++ physics engine (p_physics.cpp) reads agent positions and connection weights from SHM, runs a spring simulation until kinetic energy stabilises, then exits.
- **Active subgraph** — p_active_subgraph.prepare_active_subgraph() prunes the full graph to only the routes relevant to the two targets, repacking SHM in-place.
- **Thought traversal** — p_thought.cpp reads anchor pairs selected by p_thought_process, performs weighted random walks to build reasoning paths, and writes structured output records.
- **Synthesis** — s_synthesis.KnowledgeSynthesizer assembles cited fact blocks from the best Thought paths and sends a ~1200-word prompt to Ollama; post-processing deduplicates and formats the final report.
- **Report delivery** — The finished report string is written into the REPORT SHM segment (128 KB); the WebSocket server streams it to the dashboard at 30 Hz.

1.4 Phase State Machine

The system's lifecycle is controlled by a six-state integer stored in shared memory and managed by main.py. Each C++ subprocess reads the current phase to know when to start, and Python transitions the phase forward as each stage completes.

Phase	Description
0 — PHASE_IDLE	System waiting for a command. Dashboard shows idle state.
1 — PHASE_RESEARCH	Scraping and extraction workers are running. Graph is being built.
2 — PHASE_PHYSICS	C++ spring simulation is running. Agent positions are updating.
3 — PHASE_STABLE	Physics has converged. Active subgraph pruning occurs here.
4 — PHASE_THINKING	C++ thought engine is running. Reasoning paths are being explored.
5 — PHASE_SYNTHESIS	LLM synthesis is running. Final report is being assembled.

2. Entry & Orchestration

2.1 main.py — Process Bootstrap

main.py is the top-level entry point and process supervisor. Its responsibilities span three domains: shared memory allocation, subprocess lifecycle management, and log infrastructure.

StartupLogTee

The **StartupLogTee** class is instantiated at import time. It creates an OS-level pipe and replaces sys.stdout and sys.stderr with write ends of that pipe. A background thread drains the read end, writing each line simultaneously to the original stdout and to a timestamped log file in the logs/ directory. This ensures that all subprocess output, print statements, and exception tracebacks are persistently captured without requiring the calling shell to redirect output.

```
class StartupLogTee:
# Creates pipe, replaces sys.stdout/stderr
# Background thread: read_fd → original_stdout + log_file
# Log filename: logs/brain_YYYYMMDD_HHMMSS.log
def __init__(self): ...
def _drain(self): ... # runs in daemon thread
```

Shared Memory Allocation

main.py allocates five named POSIX shared memory segments at startup. All segment names are generated with unique suffixes via utils.create_shm() to avoid collisions with stale segments from previous runs. The segment names are exported as environment variables so that subprocesses (C++ binaries and the FastAPI server) can attach to the same regions.

Segment	Size	Contents
SHM_POS	AGENT_POSITION_RECORD_BYTES × MAX_AGENTS = 24 × 64000 = 1,536,000 bytes	Agent 3D positions (x,y,z floats × 2 copies) + index
SHM_CONNECTIONS	CONNECTION_RECORD_SIZE × MAX_CONNECTIONS + 4 = 40 × 80000 + 4 = 3,200,004 bytes	Packed 40-byte connection records + 4-byte count header
SHM_REPORT	131,072 bytes (128 KB)	UTF-8 synthesis report text, null-terminated
SHM_COMMAND	2,048 bytes	JSON or pipe-delimited command string from dashboard
SHM_STATUS	1,024 bytes	Phase integer + status string for dashboard polling

Subprocess Lifecycle

main.py manages three long-running subprocesses: the FastAPI dashboard server (frontend/ui.py via uvicorn), the C++ physics engine (venv/physics-engine), and the C++ thought engine (venv/thought-engine). The physics and thought engines are started and waited upon in sequence within the phase state machine; the dashboard server runs persistently for the entire session. All subprocesses receive the SHM segment

names via environment variables, and main.py ensures clean teardown on KeyboardInterrupt or normal exit.

The phase state machine loop in main.py polls the SHM_STATUS segment and the return codes of subprocesses to advance through phases. If the physics engine returns exit code 2 (hard timeout at 22 seconds), the system still advances to PHASE_STABLE; this prevents a permanently unstable graph from blocking synthesis.

2.2 m_runtime.py — Brain Runtime Loop

m_runtime.py hosts the **run_brain()** function, which executes as a dedicated thread inside the main process. It owns the research loop logic that sits between the phase state machine in main.py and the graph operations in p_graph.py.

Command Dispatch

run_brain() spins in a tight loop reading the COMMAND SHM segment. When a non-empty command appears, utils.parse_command_payload() decodes it to extract target_a, target_b, and optional parameters. The Brain graph object is reset, then d_query_expander.build_query_plan() is called to generate the 15-query plan. Scrape workers are launched immediately with the target-A and target-B query sets.

Ingestion Loop

While scrape and extraction workers run in parallel, run_brain() drains asu_queue in a polling loop, calling Brain.add_connection() for each result. It tracks connection counts per query category to decide whether the refinement scrape threshold has been met.

queue_refinement_scrapes()

If total extracted connections fall below a per-category threshold after the first scrape wave, queue_refinement_scrapes() constructs refined search queries by sampling the most-connected agent names found so far and combining them with the original target tokens. SCRAPE_REFINEMENT_LIMIT=16 additional pages are fetched starting at SCRAPE_REFINEMENT_OFFSET=16.

launch_thought_workers()

After the active subgraph is prepared, launch_thought_workers() calls p_thought_process.start_thought_workers(), which writes the THOUGHT_INPUT file and spawns the C++ thought engine. On completion, p_thought_process._load_cpp_thought_results() parses the output and populates the Brain object's thought list with fully reconstructed Thought instances.

prepare_active_subgraph()

This wrapper calls p_active_subgraph.prepare_active_subgraph(brain) and then logs the resulting subgraph statistics: number of agents retained, number of connections retained, and number of agents zeroed out. It also updates the phase to PHASE_STABLE.

3. Configuration

3.1 u_constants.py — System Constants

u_constants.py is a single-line-per-group flat constants file. It uses no classes or functions — every value is a module-level assignment. This makes it trivially importable in both Python worker processes and in test environments without side effects. The file is divided into logical groups by semicolon-separated assignments on single lines.

Path Configuration

Constant	Value / Expression	Purpose
PROJECT_ROOT	Path(__file__).resolve().parent	Absolute path to the repository root
SQL_DIR	PROJECT_ROOT / 'sql'	Directory for all SQLite database files
LOG_DIR	PROJECT_ROOT / 'logs'	Directory for timestamped log files
DEFAULT_MAP_PATH	SQL_DIR / 'word_info_map.sqlite3'	Semantic concept store database
SCRAPE_CACHE_PATH	SQL_DIR / 'scrape_cache.sqlite3'	Scraped page content cache
EXTRACTION_CACHE_PATH	SQL_DIR / 'extraction_cache.sqlite3'	NLP extraction result cache

Phase Constants

Constant	Value
PHASE_IDLE	0
PHASE_RESEARCH	1
PHASE_PHYSICS	2
PHASE_STABLE	3
PHASE_THINKING	4
PHASE_SYNTHESIS	5

Shared Memory Layout Constants

Constant	Value	Description
MAX_AGENTS	64,000	Maximum number of Info_Agent objects in the graph
MAX_CONNECTIONS	80,000	Maximum number of connection records in SHM
AGENT_POSITION_FLOATS	6	Floats per agent position record (x,y,z × 2 + index padding)
FLOAT_BYTES	4	Bytes per IEEE 754 single-precision float
AGENT_POSITION_RECORD_BYTES	24	Total bytes per agent position record (6 × 4)

CONNECTION_RECORD_SIZE	40	Bytes per connection record in SHM
CONNECTION_COUNT_BYTES	4	Bytes for the connection count header at start of SHM_CONNECTIONS
REPORT_SHM_BYTES	131,072	Size of the report SHM segment (128 KB)
COMMAND_SHM_BYTES	2,048	Size of the command SHM segment
STATUS_SHM_BYTES	1,024	Size of the status SHM segment

Connection Record Binary Offsets

Each 40-byte connection record is packed into SHM_CONNECTIONS as a C struct. The following offsets define the layout:

Bytes	Field	Type & Meaning
0–3	subject_id	uint32 — concept index of the subject agent
4–7	predicate_id	uint32 — concept index of the predicate/relation string
8–11	object_id	uint32 — concept index of the object agent
12–15	utility (CONNECTION_UTILITY_OFFSET=12)	float32 — computed utility score [0,1]
16–19	subject_quantifier (SUBJECT_QUANT_EXACT_OFFSET=16)	uint32 — literal index
20–23	subject_tense (SUBJECT_TENSE_EXACT_OFFSET=20)	uint32 — literal index
24–27	subject_truth (SUBJECT_TRUTH_EXACT_OFFSET=24)	uint32 — literal index
28–31	predicate_quantifier (PREDICATE_QUANT_EXACT_OFFSET=28)	uint32 — literal index
32–35	predicate_tense (PREDICATE_TENSE_EXACT_OFFSET=32)	uint32 — literal index
36–39	predicate_truth (PREDICATE_TRUTH_EXACT_OFFSET=36)	uint32 — literal index

Embedding & Merging Parameters

Constant	Value	Meaning
EMBEDDING_MODEL_NAME	all-MiniLM-L6-v2	sentence-transformers model used for all concept embeddings
MIN_MERGE_SIMILARITY	0.84	Cosine similarity floor for merging two agent names into one concept
MAX_MERGE_SIMILARITY	0.97	Cosine similarity ceiling; above this, texts are considered identical
AGENT_SEMANTIC_SEED_SCALE	420.0	Scale factor for the semantic seed 3D position projection
AGENT_SPAWN_JITTER	60.0	Random jitter added to initial agent position

AGENT_NEAR_PARENT_WEIGHT	0.15	Blend weight toward parent agent position on spawn

Worker & Capacity Limits

Constant	Value	Meaning
SCRAPE_THREADS	8	Parallel scrape worker processes
EXTRACTION_THREADS	4	Parallel extraction worker processes
THINK_THREADS	8	Parallel thought worker processes (C++ subprocess)
MAX_UI_AGENTS	2,400	Max agents sent over WebSocket per frame
MAX_UI_BONDS	3,600	Max bonds sent over WebSocket per frame
MAX_UI_CONNECTION_STATS	12,000	Max connection stat entries in WebSocket payload
WEBSOCKET_REFRESH_SECONDS	1/30 ≈ 0.033s	Target WebSocket frame interval
DASHBOARD_PORT	8000	FastAPI server listen port

Synthesis & Thinking Parameters

Constant	Value	Meaning
SYNTHESIS_MODEL	llama3.2:3b	Ollama model tag for synthesis LLM calls
SYNTHESIS_TARGET_MATCH_THRESHOLD	0.34	Min cosine similarity for a path endpoint to count as on-target
SYNTHESIS_ARGUMENT_LIMIT	3	Max thought payloads passed to one LLM synthesis call
SYNTHESIS_MAX_CHAIN_CLAUSES	6	Max clauses per reasoning chain in the synthesis prompt
ARGUMENT_SOURCE_DIVERSITY_TARGET	3	Target number of distinct source URLs per argument
ARGUMENT_SOURCE_DIVERSITY_MIN_MULTIPLIER	0.45	Min fraction of diversity target required
TARGET_SEED_LIMIT	8	Max seed agents per target during anchor selection
GOAL_AGENT_LIMIT	8	Max goal agents per target
PATH_TARGET_MATCH_THRESHOLD	0.34	Min similarity for a path step to count as on-target
MIN_SUCCESS_STEPS	2	Min steps in a thought path to be considered successful
MAX_THOUGHT_HOPS	12	Max hops allowed in a single thought path traversal
MAX_CONTEXT_SAMPLES	4	Max context samples per endpoint during subgraph expansion

ACTIVE_SUBGRAPH_CANDIDATE_MULTIPLIER	4	Candidate pool multiplier for subgraph anchor selection
ACTIVE_ANCHOR_PAIR_LIMIT	4	Max anchor pairs used for thought route selection
ACTIVE_SUBGRAPH_CONTEXT_HOPS	1	Hop radius for context expansion around selected routes
STARTING_SCORE	100.0	Initial score for every Thought path
ENDPOINT_CONTEXT_LIMIT	24	Max connections sampled per endpoint for context
THOUGHT_AGENTS_PER_WORKER	8	Thought paths launched per C++ worker thread

Scraping Parameters

Constant	Value	Meaning
SCRAPE_DEFAULT_LIMIT	16	Default number of search results to fetch per query
SCRAPE_REFINEMENT_ENABLED	True	Whether refinement scraping is attempted
SCRAPE_REFINEMENT_LIMIT	16	Result limit for refinement scrape wave
SCRAPE_REFINEMENT_OFFSET	16	Result offset (skip first 16) for refinement scrape
SERPER_RESULT_TIMEOUT	5s	HTTP timeout for Serper.dev API call
SERPER_PAGE_TIMEOUT	3s	HTTP timeout for individual page fetch
MIN_EXTRACTABLE_BLOCK_CHARS	80	Minimum characters for a text block to be worth extracting
MAX_CHARS_PER_BLOCK	2,400	Maximum characters per extraction block (truncation limit)
EXTRACTION_BLOCK_LIMIT	16	Max text blocks processed per scraped document
EXTRACTION_SENTENCE_LIMIT	96	Max sentences extracted per document
EXTRACTION_CLAUSE_LIMIT	180	Max clauses extracted per document
MAX_CONNECTIONS_PER_QUERY	512	Max connections ingested from a single query's results

3.2 u_language_constants.py — NLP Token Lists

u_language_constants.py contains large frozen sets and tuples used throughout the NLP pipeline. These are defined as module-level constants to avoid recomputing them on every call to the extraction functions. The file contains no logic — it is purely declarative data.

Constant Name (group)	Type	Purpose
STOPWORDS	frozenset	Common English stopwords; used by d_target_text to strip non-distinctive tokens
AUXILIARY_VERBS	frozenset	English auxiliary and modal verbs (is, are, was, were, have, has, will, would, can, could, shall, should, may, might, must, etc.)
QUANTIFIER_TOKENS	frozenset	Determiners and quantifiers (all, some, many, few, most, every, each, any, no, etc.)

NEGATION_CUES	frozenset	Lexical negation markers (not, never, no, neither, nor, without, etc.)
COORDINATOR_TOKENS	frozenset	Coordinating conjunctions and discourse connectives (and, or, but, yet, so, for, nor)
DISCOURSE_MARKERS	frozenset	Multi-word discourse markers (however, therefore, as a result, in contrast, etc.)
TENSE_CUES	dict/tuples	Mapping of tense cue keywords to tense label strings (past, present, future, conditional, etc.)
SUFFIX_LISTS	tuples of str	Derivational and inflectional suffix lists used by d_target_text.target_token_key() for suffix-stripping normalisation
LINKING_RELATION_PHRASES	frozenset	Multi-word phrases treated as semantic relation patterns (is a type of, is part of, contributes to, leads to, etc.)

The distinction between u_constants.py (numeric/path values) and u_language_constants.py (string token sets) is intentional: it allows NLP-heavy modules to import language constants without pulling in unrelated numeric constants, and vice versa.

4. Data Ingestion (d_ prefix)

4.1 d_scrape_worker.py — Web Scraping Worker

d_scrape_worker.py implements a multiprocessing worker function that receives a single search query string and fetches the corresponding web content. It is designed to be launched as a pool of SCRAPE_THREADS=8 parallel processes.

Search Backends

The worker supports two search backends: **Serper.dev** (primary, requires API key in environment variable SERPER_API_KEY) and **Wikipedia OpenSearch** (fallback, no key required). The Serper backend returns structured JSON with title, link, and snippet fields; the worker iterates up to SCRAPE_DEFAULT_LIMIT=16 results, fetching the full page text for each via trafilatura.

Content Extraction

trafilatura.extract() is called with include_comments=False, include_tables=False, and a short timeout to prevent hanging on slow sites. The extracted text is split into blocks at paragraph boundaries; each block must contain at least MIN_EXTRACTABLE_BLOCK_CHARS=80 characters. Blocks longer than MAX_CHARS_PER_BLOCK=2400 characters are truncated. A maximum of EXTRACTION_BLOCK_LIMIT=16 blocks per document are passed downstream.

SQLite Cache

Every fetched URL is cached in scrape_cache.sqlite3 with a namespaced key of the form '{query_hash}|{url}'. On subsequent runs with the same query, cached content is used directly without re-fetching. Cache entries include the full raw text, fetch timestamp, and HTTP status code.

Output

Scraped text blocks are placed onto a multiprocessing.Queue named extract_queue as tuples of (source_tag, content_block). The source_tag encodes the document title and URL in 'title|url' format, which is later used by utils.display_source() to produce human-readable citations.

4.2 d_extraction_worker.py — Extraction Worker

d_extraction_worker.py is the bridge between raw scraped text and structured connection records. It runs as a pool of EXTRACTION_THREADS=4 parallel processes.

The worker dequeues (source_tag, content_block) tuples from extract_queue, calls d_logic_extractor.find_connections(content_block, source_tag), and places the resulting list of connection dictionaries onto asu_queue. Each connection dict contains: subject, relation, predicate, source, evidence_text, specifics, tense, truth, quantifier, and previous_agent_ids fields.

The worker also performs simple rate-limiting: if asu_queue grows beyond a soft threshold, the worker sleeps briefly to allow the ingestion side to drain. This prevents unbounded memory growth during high-throughput scrape phases.

4.3 d_logic_extractor.py — NLP Extraction Core

This is the most algorithmically complex module in the data layer. It transforms raw text into (subject, relation, predicate) triples enriched with metadata. The extraction process has five distinct stages.

Stage 1: Text Splitting

Input text is first split into sentences using spaCy's sentence boundary detection. Each sentence is then split into clauses using a two-pass approach: first by coordinating conjunctions (identified via spaCy POS tags), then by subordinating clause markers. Each clause must have a minimum token count to be processed.

Stage 2: Candidate Triple Generation

For each clause, three extraction strategies run in parallel:

- **spaCy dependency parse**: Identifies nsubj/nsubjpass (subjects), ROOT (head verb), dobj/attr/prep (objects), and amod/advmod (modifiers). The dependency tree is traversed to assemble multi-word subjects and predicates.
- **Linking-pattern regex**: Scans clause text against the LINKING_RELATION_PHRASES set from u_language_constants. Matches like 'X is a type of Y' are directly extracted as triples.
- **Action-verb scan**: For clauses where the dependency parse yields ambiguous results, a heuristic verb-scanning pass identifies the most semantically significant verb using d_linguistic_roles.relation_word_score().

Stage 3: Metadata Enrichment

Each candidate triple is enriched with four metadata dimensions:

- **Negation detection**: Two-pass approach. First pass: lexical check against NEGATION_CUES set. Second pass: semantic check via cosine similarity of the clause embedding against a set of negation prototype embeddings. If either pass fires, truth is set to 'false'.
- **Tense parsing**: d_linguistic_roles.parsed_tense() inspects the head verb's morphological features (Tense, Aspect, Mood attributes from spaCy token.morph) and falls back to keyword lookup in TENSE_CUES if morphology is unavailable.
- **Quantifier inference**: The subject noun phrase is scanned for determiners and quantifiers from the QUANTIFIER_TOKENS set. The first matching token is used; if none is found, the quantifier defaults to 'some'.
- **Specifics extraction**: Numeric tokens (cardinal numbers, percentages, dates) in the clause are collected into a specifics dict. Dates are parsed with dateutil.parser; numbers are normalised to floats.

Stage 4: Extraction Cache

Before running the full NLP pipeline, find_connections() computes a SHA-256 hash of the concatenation of the source tag and content block. If a matching hash exists in extraction_cache.sqlite3, the cached results are returned immediately. This avoids redundant spaCy processing for identical content encountered across multiple scrape runs.

Stage 5: Output Filtering

Raw triples are filtered through d_noise_cleanup before being returned. Any triple where the subject or predicate fails is_usable_agent_text() or the relation fails is_usable_relation_text() is discarded. Coordinated subjects and predicates are split by split_coordinated_phrase() to generate additional atomic triples.

The document-level limits EXTRACTION_SENTENCE_LIMIT=96 and EXTRACTION_CLAUSE_LIMIT=180 are enforced by counters that terminate the extraction loop early if exceeded, preventing excessively long documents from monopolising extraction threads.

4.4 d_linguistic_roles.py — spaCy Wrapper

d_linguistic_roles.py wraps spaCy with a thread-safe LRU document cache to avoid re-parsing the same text multiple times during a session.

LRU Doc Cache

An 8192-entry LRU cache keyed by the input text string stores parsed spaCy Doc objects. The cache is implemented with an OrderedDict and a threading.Lock to support concurrent access from multiple extraction workers. Cache hit rate is typically very high for clause-level inputs because many clauses repeat across different documents.

Key Functions

Function	Description
dependency_relation(text)	Returns the dependency relation label (nsubj, dobj, etc.) of the head token in the parsed text.
looks_like_clause(text)	Heuristic: returns True if the text contains at least one verb-like token and at least one noun-like token, with a minimum token count.
looks_like_imperative(text)	Returns True if the first token is a base-form verb (VB tag) with no explicit subject — a common pattern in instructional text.
parsed_tense(text)	Inspects the head verb's morphological Tense and Aspect features. Returns a tense string from {past, present, future, conditional, unknown}.
embedded_statement_text(text)	Extracts the embedded clause from a reporting verb construction (e.g., 'X said that Y' → returns 'Y').
relation_word_score(token)	Computes a morphological suffix score for a verb token: base verbs score 1.0, gerunds 0.9, past-tense 0.85, passive 0.7. Used to rank relation candidates.

4.5 d_noise_cleanup.py — Text Normalization

d_noise_cleanup.py applies a battery of regex-based normalisation rules to ensure that agent names and relation strings are consistent, compact, and semantically meaningful. It enforces hard limits: MAX_AGENT_CHARS=72 characters and MAX_AGENT_TOKENS=8 whitespace-delimited tokens per agent name.

Function Summary

Function	Description
clean_text(text)	General-purpose text normaliser: collapses whitespace, strips leading/trailing punctuation, lowercases, removes HTML entities.
clean_agent_name(text)	Agent-specific normaliser: applies clean_text, then strips determiners at the start ('the', 'a', 'an'), truncates to MAX_AGENT_CHARS, truncates to MAX_AGENT_TOKENS.
clean_clause_text(text)	Clause-level normaliser: removes parenthetical asides, strips citation markers ([1], [2], etc.), normalises quotes and dashes.

is_usable_agent_text(text)	Returns True if text passes minimum quality checks: at least 2 chars, at least 1 alphabetic character, not purely numeric, not in a stop-name blacklist.
is_usable_relation_text(text)	Returns True if the relation string contains at least one alphabetic character and is not in the relation stop-list.
split_coordinated_phrase(text)	Splits a comma- or conjunction-coordinated phrase into its constituent parts. Returns a list of 1–N sub-phrases.
expand_clean_connection_record(record)	Takes a raw connection dict and applies all cleaners to its subject, relation, and predicate fields. Returns None if any field fails usability checks.

4.6 d_word_info_map.py — Semantic Store

d_word_info_map.py is the semantic backbone of the system. It manages a SQLite database (word_info_map.sqlite3) that stores sentence-transformer embeddings for all concepts, aliases, and literal strings encountered during a session.

Database Schema

Table	Columns	Purpose
concepts	id (PK), text, vector (BLOB, 384 float32s = 1536 bytes)	One row per unique concept; vector is the centroid embedding
concept_aliases	concept_id (FK), alias_text	All raw text strings that have been merged into this concept
concept_terms	concept_id (FK), term_text	Canonical term strings for display
literals	id (PK), text	Unmerged strings: relation phrases, tense labels, quantifiers, modifiers
literal_aliases	literal_id (FK), alias_text	All raw forms that map to this literal
metadata	key, value	Session metadata: embedding model name, schema version, creation timestamp

get_or_create_index()

This is the critical insertion point for all concept text. The algorithm is:

```
def get_or_create_index(text, conn=None):
1. Check 32768-entry text→index LRU cache; return hit immediately.
2. Load existing concept embeddings into numpy array (thread-local conn).
3. Encode input text with sentence-transformers model.
4. Compute cosine similarity against all existing concept vectors.
5. If best_sim >= MAX_MERGE_SIMILARITY: return existing index (near-duplicate).
6. If MIN_MERGE_SIMILARITY <= best_sim < MAX_MERGE_SIMILARITY:
merge: update centroid = merge_vectors(existing, new),
add alias, return existing index.
7. Else: INSERT new concept row, return new index.
8. Update LRU cache.
```

get_literal_index()

For strings that should NOT be merged (relation phrases, tense labels, quantifiers), get_literal_index() performs an exact string match against the literals table. If no exact match exists, a new row is inserted. No embedding or similarity computation is performed.

Vector Utilities

Function	Description
text_similarity(a, b)	Encode both texts and return cosine similarity. Cached via LRU.
cosine_similarity(v1, v2)	Pure numpy: dot(v1,v2) / (norm(v1)*norm(v2)). Handles zero-norm edge case.
vector_specificity(v)	Returns the L2 norm of a concept vector as a proxy for semantic specificity.
merge_vectors(v1, v2)	Returns (v1 + v2) / 2, normalised to unit length.

Thread Safety

SQLite connections are stored in threading.local() objects so that each worker thread uses its own connection. The LRU cache is protected by a threading.Lock. The sentence-transformers encode() call is thread-safe by default (the model is loaded once at module import time and shared via read-only access).

4.7 d_query_expander.py — Query Planning

d_query_expander.py produces a structured query plan that guides the scraping phase. The single exported function build_query_plan(target_a, target_b) returns a dict with five keys.

Key	Type	Content
target_a_queries	List of 5 strings	Search queries focused on target_a
target_b_queries	List of 5 strings	Search queries focused on target_b
bridge_queries	List of 5 strings	Queries explicitly combining both targets
focus_phrases_a	List of strings (TARGET_FOCUS_PHRASE_LIMIT=5)	Short phrases for target_a anchor selection
focus_phrases_b	List of strings	Short phrases for target_b anchor selection

The five query variants for each target are: '{target}', '{target} facts', '{target} history', '{target} causes effects', and '{target} mechanisms overview'. Bridge queries use forms like '{target_a} {target_b} relationship', '{target_a} {target_b} connection', and '{target_a} impact on {target_b}'. The constants TARGET_QUERY_LIMIT=5 and BRIDGE_QUERY_LIMIT=5 control the list lengths.

4.8 d_target_text.py — Target Tokenization

d_target_text.py provides token-level utilities for processing target query strings into forms suitable for anchor selection and similarity scoring.

Function	Description
target_tokens(text)	Lowercases and splits on whitespace; strips tokens that are in STOPWORDS. Returns list of str.

target_token_key(token)	Strips derivational suffixes (using SUFFIX_LISTS from u_language_constants) and inflectional endings. Returns the morphological root for fuzzy matching.
distinctive_target_tokens(text)	Calls target_tokens, then selects the top-3 longest remaining tokens. 'Longest' is used as a proxy for semantic specificity.
target_acronym_tokens(text)	For multi-word targets, generates all 2–4 character acronyms from the initial letters of each word. Useful for matching abbreviated references (e.g., 'machine learning' → 'ml', 'ML').

5. Object Model (o_ prefix)

5.1 o_connection.py — Connection Objects

o_connection.py defines the Python-side representation of a connection (triple) in the knowledge graph. It contains two classes: ConnectionEndpoint and Connector.

ConnectionEndpoint

ConnectionEndpoint is a lightweight __slots__ class representing one endpoint (subject or predicate side) of a connection. It stores the five metadata fields that are encoded into the binary SHM record.

```
class ConnectionEndpoint:
__slots__ = ['quantifier', 'tense', 'truth',
'modifier_idx', 'subject_predicate_id']
# quantifier: str literal (e.g. 'all', 'some', 'most')
# tense: str literal (e.g. 'past', 'present', 'future')
# truth: str literal ('true', 'false', 'uncertain')
# modifier_idx: int — literal index of adverbial modifier
# subject_predicate_id: int — concept index of agent
```

Connector

Connector wraps a 40-byte slice of the SHM_CONNECTIONS buffer. It holds both the raw binary offset into SHM and a set of Python-side metadata that is too large or complex to pack into 40 bytes.

```
class Connector:
# SHM-backed fields (read/write via struct.pack_into):
# offset: int — byte offset into SHM_CONNECTIONS
# Python-side metadata:
source: str # 'title|url' source tag
evidence_text: str # original clause text
specifics: dict # {field: value} numeric/date specifics
previous_agent_ids: list # agent indices preceding this connection
utility: float # cached utility score [0,1]
subject_ep: ConnectionEndpoint
predicate_ep: ConnectionEndpoint
```

Connector.utility_for()

utility_for(subject_agent, predicate_agent) computes the utility score for this connection by calling connector_utility() from d_word_info_map. The utility is a function of the vector specificities of both endpoint agents and the semantic distance between them. More specific and more distant agents yield higher utility, encouraging the physics engine to maintain informative long-range connections.

5.2 o_info_agent.py — Info Agent

Info_Agent represents a single node in the knowledge graph. Each agent corresponds to one concept in the semantic store, identified by its concept index.

```
class Info_Agent:
index: int # concept index in word_info_map
asu_text: str # canonical display text
info_vector: np.ndarray # 384-dim float32 embedding
pos: np.ndarray # [x, y, z] current position
connectors: list # all Connector objects attached
```

semantic_seed_position()

The initial 3D position of an agent is determined deterministically from its embedding vector via a fixed random projection matrix. This matrix is generated once using np.random.RandomState seeded by the vector dimension (384), producing a 384×3 projection matrix. The embedding vector is multiplied by this matrix and scaled by AGENT_SEMANTIC_SEED_SCALE=420.0, then a random jitter of up to AGENT_SPAWN_JITTER=60.0 units is added.

If the agent is spawned near a parent agent (indicated by the near_parent flag in p_graph._spawn_agent), the initial position is blended toward the parent's current position with weight AGENT_NEAR_PARENT_WEIGHT=0.15. This causes semantically related agents to cluster near each other before physics runs.

_vector_for_asu()

This private method retrieves the stored concept vector for the agent's index from d_word_info_map. If the index is valid but no vector is stored (e.g., for literal indices), it re-encodes the asu_text using the sentence-transformers model and caches the result.

5.3 o_thought_agent.py — Thought Tracker

The Thought class tracks one complete reasoning path through the knowledge graph, as assembled from the C++ thought engine's output.

```
class Thought:
history: list # [{agent_idx, connector_offset, step_n, ...}]
score: float # starts at STARTING_SCORE=100.0
collected_notes: list # note strings appended during traversal
collected_sources: list # source tags encountered along the path
```

_lexical_match(token, text)

Case-insensitive substring match with stemming normalisation. Checks if the token or its root form appears in the text. Used by _candidate_match_score to evaluate how closely a path step relates to the target query.

_candidate_match_score(agent, target_queries)

Scores an agent against all target query strings by combining lexical match counts with cosine similarity between the agent's info_vector and the mean embedding of the query tokens. Returns a float in [0, 1].

_path_target_matches()

Counts how many steps in the history have a match score above PATH_TARGET_MATCH_THRESHOLD=0.34 for either target_a or target_b. Used to compute the path's 'target bridge coverage' rank key for synthesis selection.

_build_relationship_statement()

This is the primary output method of the Thought class. It assembles the final payload dict that is passed to the synthesis layer. The payload includes:

Field	Type	Description
support_score	float	Weighted path score combining step match scores and specifics bonuses
confidence	str	Label derived from support_score: 'high', 'medium', or 'low'
path_clauses	list of str	Human-readable clauses describing each step in the path
clause_records	list of dict	Structured records for each step, including agent text, relation, and evidence
source_diversity	int	Count of distinct source URLs contributing to this path
source_diversity_met	bool	True if source_diversity >= ARGUMENT_SOURCE_DIVERSITY_TARGET * MIN_MULTIPLIER
target_a_coverage	float	Fraction of steps matching target_a
target_b_coverage	float	Fraction of steps matching target_b

6. Graph Layer (p_ prefix)

6.1 p_graph.py — Brain Class

The Brain class is the central data structure of the system. It owns all in-memory graph state and acts as the coordination point between the Python data layer and the C++ engine subprocesses. A single Brain instance is created in main.py and shared across threads.

State Fields

Field	Type	Description
agents	dict[str, Info_Agent]	Maps canonical agent text → Info_Agent object
agents_by_idx	dict[int, Info_Agent]	Maps concept index → Info_Agent object (for O(1) lookup by C++ index)
connection_offsets	dict[tuple, int]	Maps (subject_idx, predicate_idx, object_idx) → SHM byte offset
connection_buckets	dict[tuple, list]	Maps (subject_idx, object_idx) bucket → list of connection offset ints
connection_sources	dict[int, set]	Maps SHM offset → set of source tag strings
connection_states	dict[int, dict]	Maps SHM offset → endpoint metadata dict
connection_specifics	dict[int, dict]	Maps SHM offset → specifics dict
connection_texts	dict[int, str]	Maps SHM offset → evidence text string
connection_previous_agents	dict[int, list]	Maps SHM offset → list of preceding agent indices
thoughts	list[Thought]	All successfully completed Thought paths after thought phase
query_plan	dict	Current query plan from d_query_expander
target_a / target_b	str	Current target strings

SHM Binding

Brain.__init__() attaches to all five SHM segments by name from environment variables. It wraps each segment as a numpy array view for zero-copy access: SHM_POS becomes a (MAX_AGENTS, AGENT_POSITION_FLOATS) float32 array; SHM_CONNECTIONS becomes a uint8 array that is read/written via struct.pack_into and struct.unpack_from calls in p_connection_graph.

_spawn_agent()

Creates a new Info_Agent for a given concept index and text. Computes the semantic seed position, optionally blends toward a parent agent's position, and writes the initial position into the SHM_POS array. The agent is registered in both agents and agents_by_idx dicts.

_resolve_existing_or_mergeable_agent_name()

Before creating a new agent, this method checks whether an existing agent is semantically equivalent to the candidate name. It operates in two stages:

```
1. Exact string match: check agents dict directly.
2. Vectorized cosine similarity:
a. Encode candidate text.
b. Stack info_vectors of all merge-candidate agents (same first token).
c. Compute cosine similarity in a single numpy matmul.
d. If best_sim >= MIN_MERGE_SIMILARITY: return existing agent.
e. Else: return None (new agent needed).
```

The 'same first token' bucket is used to limit the candidate pool: only agents whose canonical text starts with the same first word are tested, reducing the O(n) cosine scan to a manageable subset.

Delegation Pattern

Brain delegates its major operations to module-level functions in companion modules: p_connection_graph.add_connection(), p_active_subgraph.prepare_active_subgraph(), p_thought_process.select_thought_routes(), and s_reporting.run_final_synthesis(). This keeps the Brain class focused on state ownership while allowing each operation to be tested and evolved independently.

6.2 p_connection_graph.py — Connection Graph

p_connection_graph.py implements the graph mutation operations. Its primary export is add_connection(brain, subject_text, relation_text, predicate_text, metadata).

add_connection() Algorithm

```
def add_connection(brain, subj, rel, pred, meta):
1. Clean subj, rel, pred via d_noise_cleanup.
2. Validate with is_usable_agent_text / is_usable_relation_text.
3. Resolve or create Info_Agent for subj and pred via brain._resolve_...
4. Get literal indices for rel, tense, quantifier, modifier, truth.
5. Compute utility via Connector.utility_for(subj_agent, pred_agent).
6. sig = _exact_connection_signature(all 11 endpoint fields).
7. If sig in connection_offsets: update utility, add source; return.
8. key = _find_compatible_connection_key(bucket).
9. If key found: merge sources into existing connection; return.
10. Pack 40-byte record via struct.pack_into into SHM_CONNECTIONS.
11. Increment SHM connection count (first 4 bytes).
12. Attach Connector to subj_agent.connectors and pred_agent.connectors.
13. Update connection_offsets, connection_buckets, connection_sources, etc.
```

_exact_connection_signature()

Encodes all 11 endpoint metadata fields (subject_id, predicate_id, object_id, subject_quantifier, subject_tense, subject_truth, predicate_quantifier, predicate_tense, predicate_truth, subject_modifier, predicate_modifier) into a Python tuple. This tuple is used as the exact-match key in connection_offsets. Two connections with identical signatures represent the same logical statement and are deduplicated.

_find_compatible_connection_key()

For a given (subject_idx, object_idx) bucket, iterates existing connections and tests whether their endpoint metadata codes are 'mutually compatible' — meaning the quantifiers, tenses, and truth values are not

contradictory. If a compatible entry is found, the new evidence is merged into it rather than creating a new connection. This reduces graph fragmentation from paraphrastic statements.

6.3 p_active_subgraph.py — Active Subgraph Pruning

After physics converges, the full graph may contain thousands of agents and connections unrelated to the two targets. p_active_subgraph.prepare_active_subgraph() prunes this to a focused subgraph.

Algorithm

```
def prepare_active_subgraph(brain):
1. Call p_thought_process.select_thought_routes() to get anchor pairs.
2. For each anchor pair (seed, goal):
a. Forward BFS from seed: collect all reachable agents.
b. Reverse BFS from goal: collect all agents that can reach goal.
c. Valid route nodes = forward_reachable ∩ reverse_reachable.
3. Union all valid route node sets → retained_agents.
4. Expand by ACTIVE_SUBGRAPH_CONTEXT_HOPS=1 hop:
add all agents directly connected to any retained_agent.
5. Also retain any connection with non-empty specifics dict.
6. Repack SHM_CONNECTIONS: write only retained connections in-place.
7. Zero SHM_POS slots for removed agents.
```

SHM Repacking

The repacking step is critical for the C++ thought engine, which reads connections directly from SHM by sequential scan. It rewrites the connection buffer starting at offset 4 (after the count header), packing only retained 40-byte records. The count header is updated to reflect the new total. All Python-side dicts (connection_offsets, connection_buckets, etc.) are rebuilt to match the new offsets.

6.4 p_thought_process.py — Thought Process

p_thought_process.py manages anchor selection and the interface to the C++ thought engine. It does not implement traversal itself — that is delegated to the C++ subprocess.

_rank_anchor_candidates()

Scores all agents in the current graph against the target query strings using a combined lexical + vector similarity metric. The top-TARGET_SEED_LIMIT=8 agents for each target are retained as seed candidates; the top-GOAL_AGENT_LIMIT=8 as goal candidates.

_connected_anchor_pairs()

Evaluates all A×B pairs of (seed, goal) candidates by computing a route score: the number of direct or 2-hop connections between them, weighted by source diversity and the presence of specific (numeric/date) evidence. Returns pairs sorted by route score descending.

select_thought_routes()

Picks ACTIVE_ANCHOR_PAIR_LIMIT=4 best (seed, goal) pairs. For each pair, builds two directed routes: A→B and B→A, to ensure bidirectional reasoning. Returns a list of route descriptor dicts that are passed to both p_active_subgraph and the C++ thought engine.

start_thought_workers()

Writes a THOUGHT_INPUT text file containing one line per route: 'seed_index spawn_count goal_index route_index'. Spawns the venv/thought-engine binary as a subprocess, passing SHM segment names via environment variables. Waits for the subprocess to complete, then calls _load_cpp_thought_results().

_load_cpp_thought_results()

Parses the THOUGHT_OUTPUT text file written by the C++ engine. The format is newline-separated records with type tags: 'thought N', 'step agent_idx connector_offset', and 'end score'. For each thought, a new Thought object is created, its history is populated from the step records, its score is set from the end record, and _build_relationship_statement() is called to produce the synthesis payload.

7. Physics & Thought Engines (C++)

7.1 p_physics.cpp — Spring Simulation

p_physics.cpp implements a real-time spring-force simulation in C++17. It is compiled into the venv/physics-engine binary and invoked as a subprocess by main.py during PHASE_PHYSICS. All communication with the Python process is via shared memory segments — no IPC messages are exchanged during the simulation.

Shared Memory Access

On startup, the binary reads the SHM segment names from environment variables (set by main.py) and attaches to SHM_POS and SHM_CONNECTIONS using shm_open() and mmap(). SHM_POS is cast to a float* array indexed as pos[agent_idx * 6 + dim]; SHM_CONNECTIONS is cast to a uint8_t* array and connection records are read via reinterpret_cast with the 40-byte struct layout.

Spring Force Model

The simulation models each connection as a spring between two agents. The spring parameters are derived from the connection's utility value (read from bytes 12–15 of the connection record as a float32):

Parameter	Specification
Equilibrium distance	Linearly interpolated between MIN_REST_DISTANCE=12.6 and MAX_REST_DISTANCE=33.6 by utility. High-utility connections have larger rest distances, spreading high-information agents apart.
Spring constant (k)	Interpolated between MIN_SPRING_STRENGTH=0.014 and MAX_SPRING_STRENGTH=0.075 by utility². Quadratic scaling makes high-utility springs significantly stiffer.
Damping factor	DAMPING=0.94 per frame. Applied as velocity *= DAMPING each frame, preventing oscillation.
Soft boundary force	A centripetal force proportional to distance from swarm centroid pushes all agents back toward the centre. Prevents the graph from dispersing to infinity.

Update Loop

```
for each frame:
1. Compute centroid = mean of all agent positions.
2. For each connection record in SHM_CONNECTIONS:
read subj_idx, obj_idx, utility.
delta = pos[obj] - pos[subj].
dist = ||delta||.
rest = lerp(MIN_REST, MAX_REST, utility).
k = lerp(MIN_K, MAX_K, utility*utility).
force = k * (dist - rest) * normalize(delta).
vel[subj] += force; vel[obj] -= force.
3. Apply soft boundary: vel[i] -= boundary_k * (pos[i] - centroid).
4. Apply damping: vel[i] *= DAMPING.
```

```
5. Update positions: pos[i] += vel[i].
6. Write updated positions back to SHM_POS.
7. Check stability: if avg KE < 0.015 for 90 frames → exit(0).
8. Check timeout: if elapsed > 22s → exit(2).
```

Exit Codes

Exit Code	Meaning
0	Normal exit — simulation converged (kinetic energy stable).
2	Timeout exit — 22 second hard limit reached without convergence.

7.2 p_thought.cpp — Thought Traversal

p_thought.cpp implements the reasoning path traversal algorithm in C++17. It is compiled into venv/thought-engine and invoked after the active subgraph is prepared.

Input Format

The engine reads THOUGHT_INPUT, a plain text file with one line per route. Each line contains four space-separated integers: seed_agent_idx, spawn_count, goal_agent_idx, and route_index. The spawn_count is typically THOUGHT_AGENTS_PER_WORKER=8.

Graph Construction

From the SHM_CONNECTIONS buffer, the engine builds two in-memory data structures for each route's goal agent:

- **Forward adjacency list**: maps each agent index to the list of (neighbor_idx, connection_offset) tuples reachable in one hop (subject→object direction).
- **Reverse BFS reachability set**: performs a BFS backward from the goal agent using the object→subject direction of the connection buffer, collecting all agents that can reach the goal in any number of hops.

choose_edge() — Weighted Random Walk

At each step of a thought path, choose_edge() selects the next hop:

```
choose_edge(current_agent, goal_idx, reachability_set, visited):
candidates = forward_adjacency[current_agent]
// Filter: prefer candidates in reachability_set and not visited
// Weight each candidate: w = 1.0 / (distance_to_goal + 1.0)
// Weighted random sample from candidates
// If all candidates visited: allow revisit with lower weight
// If no candidates: return -1 (dead end)
```

append_goal_route_tail()

Once the traversal reaches the goal or exhausts MAX_THOUGHT_HOPS=12 steps, append_goal_route_tail() extends the path deterministically by following the highest-weight edge at each step, without randomness. This produces a canonical tail that is reproducible and serves as a consistent anchor for the reasoning chain.

Output Format

For each completed thought path, the engine writes to THOUGHT_OUTPUT:

```
thought <N> // start of thought N
step <agent_idx> <conn_offset> // one line per hop
...
end <score> // final score float
```

The score is computed by the C++ engine as a weighted sum of hop count and on-target step fraction. Python-side re-scoring happens in Thought._build_relationship_statement().

8. Synthesis & Reporting (s_ prefix)

8.1 s_synthesis.py — Knowledge Synthesizer

s_synthesis.py contains the KnowledgeSynthesizer class, which is responsible for transforming a set of structured Thought payloads into a coherent natural-language report via a locally-hosted LLM.

synthesize_relationship_report()

This is the sole public method of KnowledgeSynthesizer. It takes a list of thought payload dicts (output of Thought._build_relationship_statement()) and target strings, and returns a formatted report string.

Payload Selection

From the full list of available payloads (potentially dozens of thought paths), synthesize_relationship_report() selects a diverse subset of up to SYNTHESIS_ARGUMENT_LIMIT=3. Diversity is enforced by maximising source URL overlap: the second and third payloads are chosen to contribute new source URLs not already covered by the first. The goal is ARGUMENT_SOURCE_DIVERSITY_TARGET=3 distinct sources.

Source Map Construction

All unique source tags from the selected payloads are collected and assigned sequential citation numbers [1], [2], [3], etc. The source_map dict maps each tag to its number. utils.display_source() is called to extract the human-readable URL from 'title|url' tags.

Prompt Assembly

A structured ~1200-word prompt is assembled with the following sections:

- System instruction: instructs the model to write a factual analytical report without filler phrases, speculation, or meta-commentary.
- Fact blocks: for each selected payload, a numbered block of cited facts derived from the clause_records, with inline [N] citation markers.
- Chain clauses: up to SYNTHESIS_MAX_CHAIN_CLAUSES=6 path_clauses per payload, formatted as a numbered reasoning chain.
- Output instructions: specifies the required output format (paragraph-form prose, each sentence cited, no headers, no bullets).

LLM Call

The prompt is sent to the Ollama API at OLLAMA_GENERATE_URL='http://localhost:11434/api/generate' using the requests library with connect timeout DEFAULT_OLLAMA_CONNECT_TIMEOUT=5s and read timeout DEFAULT_OLLAMA_READ_TIMEOUT=300s. The response is streamed and accumulated. If Ollama is not available, a fallback report is generated directly from the fact blocks without LLM processing.

Post-Processing Pipeline

The raw LLM output passes through a multi-stage cleaning pipeline:

Step	Description
Strip template headings	Removes any lines that look like section headers (all-caps, or starting with ##, or matching known template patterns).

Strip structure language	Removes phrases like 'Target A', 'Target B', 'evidence suggests', 'it can be concluded', 'in summary', 'in conclusion' that indicate the model is narrating its own structure.
Ensure sentence citations	Any sentence that contains a factual claim (identified by presence of a proper noun or numeric token) but lacks a [N] citation has the most appropriate citation appended.
Jaccard deduplication	Sentences are compared pairwise by their set of 'claim tokens' (nouns + numbers). Sentences with Jaccard similarity above 0.6 are considered duplicates; only the longer one is kept.
Compact source numbering	After deduplication, some citation numbers may be unused. The source references are renumbered compactly [1], [2], ... and all inline citations are updated to match.
Append source list	A formatted source list is appended at the end of the report, with each source on its own line as '[N] URL'.

8.2 s_reporting.py — Final Reporting

s_reporting.py orchestrates the final reporting phase, sitting above KnowledgeSynthesizer and below the SHM write.

run_final_synthesis()

Selects the top-N Thought objects from brain.thoughts by a multi-key rank function. The rank key (in descending priority order) is:

```
rank_key(thought) = (
target_bridge_coverage, # primary: fraction of steps on both targets
path_concreteness, # secondary: fraction of steps with specifics
source_count, # tertiary: number of distinct sources
support_score # quaternary: raw Thought score
)
```

The top-ranked Thought payloads are passed to KnowledgeSynthesizer.synthesize_relationship_report(). The returned report string is written to SHM_REPORT via utils.write_report().

write_argument_report()

Writes a human-readable argument_report.txt to the project root for debugging. Contains, for each Thought path: the path_clauses list, support_score, source_diversity, and a dump of the clause_records. This file is useful for understanding why particular reasoning paths were selected or rejected.

export_final_results()

Creates a timestamped directory under RESULTS_DIR='results/'. Writes three files:

- **synthesis.txt**: The final report text.
- **connections.txt**: A formatted dump of all connections in the active subgraph, sorted by utility descending.
- **arguments.txt**: A copy of the argument_report.txt content for archival.

The results directory path is stored in brain for later retrieval by the FastAPI /results endpoint.

9. Dashboard & Frontend

9.1 frontend/ui.py — FastAPI Server

ui.py is a FastAPI application that serves as the bridge between the Brain process and the web-based dashboard. It runs as a separate subprocess launched by main.py via uvicorn, sharing the same SHM segments via environment variables.

SHM Access

On startup, ui.py reads the same five SHM segment names from environment variables and attaches to them using the same mmap-based approach as the main process. This gives the FastAPI server direct read access to agent positions, connection data, the current phase, and the report text — without any inter-process messaging overhead.

WebSocket /ws

The primary data stream endpoint. On connection, it enters a loop that fires every WEBSOCKET_REFRESH_SECONDS=1/30 seconds (30 Hz target). Each frame assembles a JSON payload containing:

Key	Contents
agents	Up to MAX_UI_AGENTS=2,400 agent records: {idx, text, x, y, z, degree}
bonds	Up to MAX_UI_BONDS=3,600 bond records: {a_idx, b_idx, utility, tinted_color}
phase	Current phase integer (0–5)
report	The current contents of SHM_REPORT (may be empty string if not yet synthesised)
connection_stats	Up to MAX_UI_CONNECTION_STATS=12,000 records for the connection detail panel

Agent and bond counts are capped to prevent the WebSocket payload from exceeding browser processing limits. When the graph is larger than the caps, agents and bonds are sampled by degree (most-connected first) to ensure the most important nodes are always visible.

POST /command

Accepts a JSON body with target_a and target_b strings. Validates that the current phase is PHASE_IDLE (phase == 0) before accepting the command — commands submitted during active phases are rejected with a 409 Conflict response. On acceptance, the command JSON string is written into the SHM_COMMAND segment.

GET /results and GET /results/{id}

Serves the history of synthesis results from the RESULTS_DIR directory. /results returns a list of available result directories sorted by timestamp. /results/{id} returns the synthesis.txt content for a specific result, enabling the dashboard's history sidebar to display past reports.

9.2 frontend/main.js — Three.js Visualization

main.js implements the real-time 3D knowledge graph visualization using Three.js r155, imported as an ES module from CDN. It establishes a WebSocket connection to ui.py and renders the graph at up to 30 fps.

Agent Rendering

Agents are rendered as SphereGeometry meshes with MeshPhongMaterial. The colour of each agent is determined by its degree (connection count) via a log-normalised gradient from purple (low degree) to green (high degree). Agents are sized by a fixed radius of 1.2 units. A single merged BufferGeometry is used for all agents to reduce draw calls.

Bond Rendering

Connections are rendered as LineSegments. Each bond has a colour that is a blend between the agent colours of its two endpoints, tinted by the connection utility value: high-utility bonds are slightly brighter. All bond geometry is packed into a single Float32Array and updated in-place each frame to avoid garbage collection pauses.

Camera Auto-Orbit

The camera automatically orbits around the graph. The orbit radius is set to the 95th percentile of all agent distances from the centroid, ensuring the camera always encompasses the graph without being dominated by outlier positions. The camera altitude oscillates sinusoidally between -30° and +30° over a 40-second cycle to provide a 3D perspective.

Chat UI

The right panel implements a chat-style interface. User input (target A/B) is submitted via a form POST to /command. The report text, once available in the WebSocket stream, is rendered as formatted HTML with citation links highlighted. A copy-to-clipboard button copies the raw report text.

Phase Debug Panel

A collapsible debug panel at the bottom of the screen shows the current phase as an icon (idle: circle, research: magnifying glass, physics: atom, etc.) and a progress rail that fills as the phase advances. Per-phase elapsed time is displayed to help diagnose performance bottlenecks.

History Sidebar

A sidebar lists past synthesis results by polling /results every 15 seconds. Clicking a result fetches /results/{id} and displays the archived report in the right panel.

9.3 frontend/index.html — UI Shell

index.html is a single-page application shell. It defines the DOM structure for the two-column layout: a full-height Three.js canvas on the left occupying approximately 60% of the viewport width, and a report panel on the right containing the input form, phase indicators, and report display area.

Three.js is imported from CDN as an ES module (<script type='module'>), which requires the page to be served from an HTTP server — opening the file directly from the filesystem will fail due to CORS restrictions on module imports. The FastAPI server handles this by serving the frontend/ directory as a static files mount.

The input section contains two text fields (Target A, Target B) and a 'Launch' button. On click, the button is disabled and a spinner is shown until the phase returns to PHASE_IDLE after synthesis completes.

10. Utilities

10.1 utils.py — Shared Utilities

utils.py is a flat module of shared helper functions used across the codebase. It has no classes and minimal state. All functions are imported at the top of the modules that need them.

Function	Description
remove_shm_from_resource_tracker()	Monkey-patches Python's multiprocessing.resource_tracker to suppress the ResourceWarning that fires when SHM segments created in one process are accessed from another. This is necessary because Python's tracker incorrectly flags cross-process SHM access as a leak.
parse_command_payload(raw)	Parses a raw command string from SHM_COMMAND. Supports two formats: JSON ({"target_a": ..., "target_b": ...}) and pipe-delimited (target_a|target_b). Returns a dict with target_a, target_b, and optional parameters.
flush_conn_log()	Flushes the internal connection log buffer to disk. Called periodically during the research phase.
clear_report(shm_report)	Writes a null byte to the start of the SHM_REPORT segment, effectively clearing the report display on the dashboard.
write_report(shm_report, text)	Encodes text as UTF-8, truncates to REPORT_SHM_BYTES-1, and writes into SHM_REPORT with a null terminator.
ensure_port_free(port)	Uses lsof to find any process listening on the given port. If found, sends SIGTERM to free the port before starting the dashboard server. Includes a short sleep to allow the OS to reclaim the port.
create_shm(size)	Creates a POSIX shared memory segment with a unique name (using uuid4). Returns (shm_object, name) tuple. The name is exported as an environment variable for subprocess access.
display_source(source_tag)	Extracts the URL portion from a 'title|url' source tag. If the tag does not contain '|', returns the tag unchanged.
merge_specifics(a, b)	Merges two specifics dicts by combining their value lists for matching keys. Used when two connections are merged in p_connection_graph.
format_specifics(specifics)	Formats a specifics dict as a human-readable string for inclusion in evidence text. Example: '{year: 1990, value: 45.2%}'.

11. Cross-Cutting Concerns

11.1 Shared Memory Layout

The Brain system uses POSIX shared memory (shm_open / mmap) as its primary IPC mechanism. This allows the Python main process, the FastAPI subprocess, and the C++ engine subprocesses to share large data arrays without serialisation overhead.

SHM_POS Layout

SHM_POS contains MAX_AGENTS=64,000 position records, each 24 bytes = 6 × float32. The layout per agent at index i:

Byte Offset	Type	Description
i*24 + 0	float32	x position (current)
i*24 + 4	float32	y position (current)
i*24 + 8	float32	z position (current)
i*24 + 12	float32	x position (previous frame, used by physics for velocity)
i*24 + 16	float32	y position (previous frame)
i*24 + 20	float32	z position (previous frame)

SHM_CONNECTIONS Layout

SHM_CONNECTIONS begins with a 4-byte little-endian uint32 connection count, followed by up to MAX_CONNECTIONS=80,000 connection records of 40 bytes each. The binary layout of each record is detailed in Section 3.1 (Connection Record Binary Offsets).

SHM_REPORT and SHM_COMMAND

Both of these segments are treated as null-terminated UTF-8 string buffers. The writer encodes the string, writes the bytes, and appends a null byte. The reader scans for the null terminator and decodes. Because only one writer exists for each segment (m_runtime writes COMMAND; s_reporting writes REPORT), no explicit locking is needed — the null-terminator convention provides a coarse form of atomic visibility.

11.2 SQLite Databases

The system uses three SQLite databases, all stored in the sql/ directory:

Database	Persistence	Purpose
word_info_map.sqlite3	Persistent across runs	Core semantic store. Concept embeddings, aliases, literals. Grows across sessions as new concepts are encountered. Should be cleared when switching between unrelated domains.
scrape_cache.sqlite3	Persistent across runs	Cached raw page text. Indexed by {query_hash}|{url} key. Prevents redundant HTTP fetches for recently-seen queries.
extraction_cache.sqlite3	Persistent across runs	Cached extraction results. Indexed by SHA-256 of source+content. Prevents re-running spaCy on identical text blocks.

All three databases are accessed with thread-local connections to avoid SQLite's single-writer constraint: each worker thread opens its own connection. WAL mode is enabled on word_info_map.sqlite3 to allow concurrent reads from the dashboard thread while extraction workers write.

11.3 Concurrency Model

The Brain system uses a layered concurrency model combining multiprocessing, threading, and subprocess management:

Component	Concurrency Role
Main process (Python)	Single process; owns all SHM; runs run_brain() thread and phase state machine loop.
run_brain() thread	Background thread in main process; owns research loop, command polling, queue draining.
Scrape workers	8 parallel multiprocessing.Process instances; each handles one query; terminate after results placed on queue.
Extraction workers	4 parallel multiprocessing.Process instances; persistent during research phase; drain extract_queue.
FastAPI server	Separate subprocess (uvicorn); shares SHM read-only for position and connection data; accepts commands write-only to SHM_COMMAND.
Physics engine (C++)	Short-lived subprocess; reads/writes SHM_POS; reads SHM_CONNECTIONS; exits when stable or timeout.
Thought engine (C++)	Short-lived subprocess; reads SHM_CONNECTIONS and THOUGHT_INPUT file; writes THOUGHT_OUTPUT file; exits when all paths complete.

The GIL (Global Interpreter Lock) is not a bottleneck because the CPU-intensive work (spaCy parsing, sentence-transformers encoding) is offloaded to worker processes, not threads. The main thread only does light bookkeeping (dict updates, SHM writes) and can yield the GIL freely.

11.4 Connection Record Binary Format — Deep Dive

The 40-byte connection record is the system's most performance-critical data structure, accessed by both the Python layer (via struct.pack_into/unpack_from) and the C++ engines (via reinterpret_cast to a packed struct). Its layout must be exactly consistent between the two languages.

In C++, the struct is defined as:

```
#pragma pack(push, 1)
struct ConnectionRecord {
uint32_t subject_id; // bytes 0-3
uint32_t predicate_id; // bytes 4-7
uint32_t object_id; // bytes 8-11
float utility; // bytes 12-15
uint32_t subj_quantifier; // bytes 16-19
uint32_t subj_tense; // bytes 20-23
uint32_t subj_truth; // bytes 24-27
uint32_t pred_quantifier; // bytes 28-31
```

```
uint32_t pred_tense; // bytes 32-35
uint32_t pred_truth; // bytes 36-39
};
#pragma pack(pop)
static_assert(sizeof(ConnectionRecord) == 40, "layout mismatch");
```

In Python, the equivalent struct format string is '<IIIfIIIII' (little-endian, three uint32, one float32, six uint32). The format characters map to: I=uint32 (4 bytes), f=float32 (4 bytes). Total: 10 × 4 = 40 bytes.

The utility field at offset 12 is special: it is the only floating-point field in the record. Both the physics engine (which uses it as spring strength) and the thought engine (which uses it as edge weight) read it as a float32. Python writes it with struct.pack_into using the 'f' format character.

Index values (subject_id, predicate_id, object_id) are concept indices from d_word_info_map. Quantifier, tense, and truth fields are literal indices from d_word_info_map.get_literal_index(). Index 0 is reserved as 'null/unknown' for unset metadata fields.

End of Brain Codebase Technical Reference

12. Appendix

12.1 End-to-End Request Trace

This section traces a single research request from the moment the user clicks 'Launch' in the browser to the moment the synthesised report appears on screen. The trace covers every module boundary and data transformation.

Step	Module	Action
1	`Browser`	User fills Target A='climate change', Target B='agriculture'. Clicks Launch.
2	`frontend/index.html`	JavaScript handler fires. Disables Launch button. POSTs JSON {target_a, target_b} to /command.
3	`frontend/ui.py POST /command`	Validates phase==PHASE_IDLE. Calls utils.write_report(shm_command, json_string). Returns HTTP 200.
4	`m_runtime.run_brain() thread`	Polls SHM_COMMAND. Detects non-empty string. Calls utils.parse_command_payload() → {target_a, target_b}.
5	`d_query_expander.build_query_plan()`	Produces 5 target-A queries, 5 target-B queries, 5 bridge queries, and two focus phrase lists.
6	`main.py phase state machine`	Transitions phase 0→1 (PHASE_RESEARCH). Writes phase int to SHM_STATUS.
7	`d_scrape_worker ×8 processes`	Each process receives one query. Calls Serper.dev API. Fetches up to 16 pages via trafilatura. Splits into blocks. Writes blocks to extract_queue.
8	`d_extraction_worker ×4 processes`	Dequeues blocks. Calls d_logic_extractor.find_connections(). Pushes connection dicts to asu_queue.

9	`m_runtime ingestion loop`	Drains asu_queue. For each connection dict, calls brain.add_connection() → p_connection_graph.add_connection(). Updates SHM_CONNECTIONS count.
10	`d_word_info_map.get_or_create_index()`	For each agent name: encodes with sentence-transformers, cosine-compares against existing concepts, merges or creates, returns concept index.
11	`p_connection_graph.add_connection()`	Packs 40-byte struct into SHM_CONNECTIONS. Attaches Connector objects to both endpoint agents.
12	`m_runtime.queue_refinement_scrapes()`	If connections < threshold, launches second wave of 16 more scrapes with refined queries derived from top agents.
13	`main.py phase state machine`	All workers done. Transitions phase 1→2 (PHASE_PHYSICS). Spawns venv/physics-engine subprocess.
14	`p_physics.cpp`	Reads SHM_POS + SHM_CONNECTIONS. Runs spring simulation until KE stable or 22s timeout. Writes final positions to SHM_POS. Exits.
15	`main.py`	physics-engine exits. Transitions phase 2→3 (PHASE_STABLE).
16	`p_active_subgraph.prepare_active_subgraph()`	BFS reachability from seed→goal pairs. Retains only on-route agents + context hop. Repacks SHM_CONNECTIONS in-place.
17	`main.py`	Transitions phase 3→4 (PHASE_THINKING). Spawns venv/thought-engine subprocess.
18	`p_thought_process.start_thought_workers()`	Writes THOUGHT_INPUT file with anchor pair routes. Waits for thought-engine to complete.
19	`p_thought.cpp`	Reads SHM_CONNECTIONS + THOUGHT_INPUT. Performs weighted random walks. Writes THOUGHT_OUTPUT records.
20	`p_thought_process._load_cpp_thought_results()`	Parses THOUGHT_OUTPUT. Reconstructs Thought objects. Calls _build_relationship_statement() on each.
21	`main.py`	Transitions phase 4→5 (PHASE_SYNTHESIS).
22	`s_reporting.run_final_synthesis()`	Ranks Thought objects. Calls KnowledgeSynthesizer.synthesize_relationship_report().
23	`s_synthesis.KnowledgeSynthesizer`	Assembles fact blocks + chain clauses. Sends ~1200-word prompt to Ollama /api/generate. Streams response. Post-processes output.
24	`utils.write_report()`	Encodes final report as UTF-8. Writes into SHM_REPORT with null terminator.
25	`s_reporting.export_final_results()`	Creates timestamped results/ directory. Writes synthesis.txt, connections.txt, arguments.txt.
26	`main.py`	Transitions phase 5→0 (PHASE_IDLE).
27	`frontend/ui.py WebSocket /ws`	Next 30Hz frame reads SHM_REPORT (non-empty). Includes report string in JSON payload to browser.
28	`frontend/main.js`	WebSocket handler receives payload. Renders report HTML in right panel. Re-enables Launch button.

12.2 Module Dependency Graph

The table below enumerates the direct import dependencies between modules. This helps identify the correct layer boundaries and prevent circular imports.

Module	Direct Imports
main.py	m_runtime, p_graph, utils, u_constants, frontend/ui (subprocess)
m_runtime.py	p_graph, p_active_subgraph, p_thought_process, d_query_expander, d_scrape_worker, d_extraction_worker, s_reporting, u_constants, utils
u_constants.py	(none — leaf module)
u_language_constants.py	(none — leaf module)
d_scrape_worker.py	d_noise_cleanup, u_constants, utils
d_extraction_worker.py	d_logic_extractor, d_noise_cleanup, u_constants
d_logic_extractor.py	d_linguistic_roles, d_noise_cleanup, d_word_info_map, u_constants, u_language_constants
d_linguistic_roles.py	u_language_constants
d_noise_cleanup.py	u_constants, u_language_constants
d_word_info_map.py	u_constants
d_query_expander.py	d_target_text, u_constants
d_target_text.py	u_language_constants
o_connection.py	d_word_info_map, u_constants
o_info_agent.py	d_word_info_map, u_constants
o_thought_agent.py	d_word_info_map, u_constants
p_graph.py	o_connection, o_info_agent, o_thought_agent, p_connection_graph, p_active_subgraph, p_thought_process, s_reporting, u_constants, utils
p_connection_graph.py	o_connection, o_info_agent, d_word_info_map, d_noise_cleanup, u_constants
p_active_subgraph.py	p_thought_process, u_constants
p_thought_process.py	o_thought_agent, d_word_info_map, d_target_text, u_constants
s_synthesis.py	d_word_info_map, utils, u_constants
s_reporting.py	s_synthesis, o_thought_agent, utils, u_constants
frontend/ui.py	utils, u_constants
utils.py	u_constants

12.3 Configuration Quick-Reference

The following table provides a consolidated quick-reference for the most commonly tuned parameters. All values come from u_constants.py unless noted.

Parameter	Default	Tuning Notes
EMBEDDING_MODEL_NAME	all-MiniLM-L6-v2	Change to a larger model for higher quality at the cost of speed
MIN_MERGE_SIMILARITY	0.84	Lower to merge more aggressively; raise to keep more distinct concepts
MAX_MERGE_SIMILARITY	0.97	Upper bound; texts above this are treated as identical duplicates
SCRAPE_THREADS	8	Increase if bandwidth allows; decrease to reduce Serper API usage
EXTRACTION_THREADS	4	Increase on multi-core machines; each thread runs a spaCy instance
SCRAPE_DEFAULT_LIMIT	16	Increase for denser graphs; each unit adds ~1 web page fetch
SYNTHESIS_ARGUMENT_LIMIT	3	Increase for richer reports; each unit adds one LLM fact block
MAX_THOUGHT_HOPS	12	Decrease for shorter, more focused chains; increase for broader paths
SYNTHESIS_MODEL	llama3.2:3b	Swap for a larger model (llama3.2:8b) for higher-quality prose
ACTIVE_ANCHOR_PAIR_LIMIT	4	Increase to explore more diverse reasoning routes
PATH_TARGET_MATCH_THRESHOLD	0.34	Lower to accept weaker on-target evidence; raise to be stricter
DASHBOARD_PORT	8000	Change if port is in use; must match browser URL

12.4 Error Handling and Failure Modes

The Brain system is designed to be fault-tolerant at every layer. The following table documents known failure modes and how each is handled.

Failure Mode	Handling Strategy
Serper API key missing	d_scrape_worker falls back to Wikipedia OpenSearch for all queries. Quality degrades but system continues.
Serper page fetch timeout (3s)	Page is skipped. SERPER_PAGE_TIMEOUT=3s prevents individual slow pages from blocking the worker.
trafilatura returns None	Block is skipped. No extraction attempted. Logged at DEBUG level.
spaCy parse error	Clause is skipped. Outer try/except in find_connections() catches all parser exceptions.
SQLite write conflict	Thread-local connections + WAL mode prevent conflicts. If a write fails, the connection record is silently dropped.
Physics timeout (22s)	p_physics.cpp exits with code 2. main.py treats exit code 2 identically to 0 (success) and advances to PHASE_STABLE.

Ollama not available	KnowledgeSynthesizer catches ConnectionRefusedError and generates a fallback report from raw fact blocks without LLM prose.
SHM_REPORT overflow	utils.write_report() truncates text to REPORT_SHM_BYTES-1=131071 bytes before writing. Long reports are silently truncated.
No thought paths found	s_reporting.run_final_synthesis() returns an empty-report sentinel. The dashboard displays a 'no results' message.
Duplicate command during research	POST /command returns HTTP 409 if phase != PHASE_IDLE. The current research is not interrupted.
Agent name too long	d_noise_cleanup.clean_agent_name() truncates to MAX_AGENT_CHARS=72. If still invalid, is_usable_agent_text() rejects it.
Concept vector store grows too large	No automatic pruning. User must manually delete word_info_map.sqlite3 to reset the semantic store.

12.5 Glossary

Key terms used throughout this document and the codebase:

Term	Definition
Agent / Info_Agent	A single node in the knowledge graph, corresponding to one semantic concept. Identified by a concept index.
ASU text	Agent, Subject, or Object text — the canonical display string for an agent. 'ASU' = Agent Semantic Unit.
Concept index	An integer primary key in the word_info_map.sqlite3 concepts table. Unique per merged concept.
Connection / Connector	A directed edge in the knowledge graph, representing a (subject, relation, predicate) triple with metadata.
Connection record	The 40-byte binary representation of a Connector in SHM_CONNECTIONS.
Literal index	An integer primary key in the literals table. Used for unmerged strings: tense labels, quantifiers, relation phrases.
Phase	One of six integer states (0–5) describing the current stage of the research session.
SHM	POSIX shared memory. A memory-mapped region accessible from multiple processes without serialisation.
Thought / Thought path	A complete reasoning chain through the knowledge graph, assembled by the C++ thought engine.
Active subgraph	The pruned subset of the full graph retained for thought traversal — only routes relevant to the two targets.
Anchor pair	A (seed agent, goal agent) pair selected by p_thought_process to define one reasoning route.
Utility	A float in [0,1] scoring a connection's informativeness. Higher utility = stiffer spring + higher traversal weight.

Semantic seed position	A deterministic 3D coordinate derived from an agent's embedding vector via random projection.
Specifics	Numeric and date values extracted from evidence text. Stored in the connection's specifics dict.
Source tag	A 'title|url' string identifying the web document from which a connection was extracted.
Target bridge coverage	The fraction of steps in a Thought path that match both target_a and target_b. Primary synthesis rank key.
WAL mode	Write-Ahead Logging — a SQLite journal mode enabling concurrent readers during writes.
Cosine similarity	A measure of vector similarity computed as dot(v1,v2)/(||v1||*||v2||). Used for concept merging and agent matching.

12.6 Build and Deployment Notes

The Brain system requires a specific build and runtime environment. The following notes summarise the setup process.

Python Environment

A Python 3.10+ virtual environment is required. Key dependencies include: spaCy (with en_core_web_sm model), sentence-transformers, FastAPI, uvicorn, trafilatura, requests, and numpy. The u_requirements.txt file lists all pinned versions. The virtual environment is expected at venv/ relative to PROJECT_ROOT.

C++ Build

The physics and thought engines are compiled with build.sh using g++ -O3 -std=c++17. The output binaries are placed at venv/physics-engine and venv/thought-engine. No external C++ dependencies are required beyond the standard library and the POSIX SHM headers (sys/mman.h). On macOS, shm_open requires linking with no extra flags; on Linux, -lrt may be required.

Environment Variables

Variable	Purpose
SERPER_API_KEY	Required for web search. Get from serper.dev. Without this, only Wikipedia fallback is used.
SHM_POS_NAME	Set by main.py at runtime. Points to the position SHM segment. Read by C++ engines and ui.py.
SHM_CONNECTIONS_NAME	Set by main.py at runtime. Points to the connections SHM segment.
SHM_REPORT_NAME	Set by main.py at runtime. Points to the report string SHM segment.
SHM_COMMAND_NAME	Set by main.py at runtime. Points to the command string SHM segment.
SHM_STATUS_NAME	Set by main.py at runtime. Points to the phase/status SHM segment.

Running the System

```
# 1. Install Python dependencies
pip install -r u_requirements.txt
```

```
python -m spacy download en_core_web_sm

# 2. Build C++ engines
bash build.sh

# 3. Set Serper API key
export SERPER_API_KEY=your_key_here

# 4. Start Ollama with the synthesis model
ollama pull llama3.2:3b
ollama serve &

# 5. Run Brain
python main.py

# 6. Open dashboard
open http://localhost:8000
```

12.7 Performance Characteristics

The following table provides empirical timing estimates for each phase of a typical research session. These figures are for a mid-range developer laptop (M1 MacBook Pro, 16 GB RAM) with a fast internet connection and all caches cold.

Phase	Module	Typical Time	Bottleneck
Query expansion	d_query_expander	<0.1s	Pure string manipulation; no I/O.
Scraping (first wave)	d_scrape_worker ×8	15–45s	Network-bound. Serper API latency + page fetches.
Extraction (first wave)	d_extraction_worker ×4	20–60s	CPU-bound. spaCy + sentence-transformers per clause.
Scraping (refinement)	d_scrape_worker ×8	10–30s	Same as first wave but smaller result set.
Graph ingestion	p_connection_graph	1–5s	SQLite reads + numpy cosine comparisons.
Physics simulation	p_physics.cpp	2–22s	Terminates early when KE stable; worst case 22s timeout.
Active subgraph pruning	p_active_subgraph	<1s	BFS + SHM repack; bounded by connection count.
Thought traversal	p_thought.cpp	1–8s	Depends on graph density and MAX_THOUGHT_HOPS.
LLM synthesis	s_synthesis + Ollama	30–120s	Model-dependent. llama3.2:3b on CPU: ~60s typical.
Total session	All phases	80–300s	Typical range; heavily network and model dependent.

The most significant optimisation levers are: (1) SCRAPE_THREADS — more threads reduce wall-clock time for the network-bound scraping phase; (2) SYNTHESIS_MODEL — a quantised or GPU-accelerated model reduces synthesis time dramatically; (3) cold vs. warm SQLite caches — on subsequent runs with similar topics, extraction and word_info_map lookups are significantly faster due to caching.